Sahayak AI: A Multimodal Agentic Retrieval-Augmented Generation Platform for Personalized Learning

Shikher Jain
Department of Computer Science and Engineering
Agra College, Dr. A.P.J. Abdul Kalam Technical University (AKTU)
Agra, Uttar Pradesh, India
shikherjain786@gmail.com | github.com/Shikher-jain

***Abstract*—Large Language Models (LLMs) demonstrate remarkable natural language capabilities, yet their tendency to hallucinate limits deployment in knowledge-intensive educational settings. Retrieval-Augmented Generation (RAG) mitigates this by grounding LLM outputs in verified external knowledge, but existing systems are restricted to single-modality textual inputs and lack learner-adaptive personalisation. This paper presents Sahayak AI, a production-grade multimodal agentic RAG platform for personalised learning. Sahayak AI ingests PDFs, images (OCR), audio (Whisper ASR), video, source code (AST-chunking), and CSV/Excel files; indexes content in a fault-tolerant hybrid vector store (Qdrant primary, FAISS/SQLite offline fallback); and synthesises grounded answers via a cascaded LLM chain (Groq llama3-70b, OpenAI GPT, HuggingFace Flan-T5). The agentic pipeline incorporates query rewriting, modality-aware semantic and recursive chunking, cross-encoder re-ranking, LangChain conversational memory, and role-conditioned prompt engineering supporting student, teacher, and general interaction modes. Empirical evaluation demonstrates a 34 % improvement in retrieval Precision@5 and consistent sub-200 ms end-to-end latency over naive RAG baselines. The system is open-source at github.com/Shikher-jain/SAHAYAK_AI.**

***Index Terms*—Retrieval-Augmented Generation, Large Language Models, Multimodal Learning, Vector Databases, Semantic Search, Agentic AI, Educational Technology, Natural Language Processing, FastAPI, LangChain.**

I. Introduction

The proliferation of digital learning content—spanning lecture recordings, scanned PDFs, programming tutorials, and tabular datasets—has created urgent demand for intelligent retrieval systems capable of unifying heterogeneous knowledge sources into a coherent, queryable interface. Conventional search engines rely on keyword overlap and fail to capture semantic intent, while standard LLMs suffer from hallucination on domain-specific queries beyond their training distribution [1].

Retrieval-Augmented Generation (RAG) [2] mitigates hallucination by conditioning LLM generation on retrieved passages from an external corpus. Early RAG deployments assume clean textual inputs and single-turn interactions, leaving a critical gap for real-world educational systems requiring: (i) multimodal ingestion, (ii) multi-turn conversational context, (iii) role-adaptive responses, and (iv) offline-first operation.

This paper makes the following **contributions**:

- A unified multimodal ingestion pipeline processing PDFs, images (OCR), audio (Whisper ASR), video, source code (AST-chunking), and CSV/Excel into a common embedding space.
- An agentic RAG architecture with query rewriting, modality-aware semantic and recursive chunking, cross-encoder re-ranking, and LangChain sliding-window conversational memory.
- A fault-tolerant hybrid vector store: Qdrant (cloud primary) plus FAISS/SQLite (offline fallback) with transparent auto-switching.
- A cascaded LLM chain (Groq llama3-70b, OpenAI GPT, HuggingFace Flan-T5) for cost-optimised, availability-resilient generation.
- Role-conditioned prompt engineering for student, teacher, and general modes with automatic language detection across five languages.
- A production FastAPI backend (25+ endpoints), JWT/OAuth2 RBAC, Docker Compose orchestration, and Streamlit interactive frontend.

II. Related Work

A. Retrieval-Augmented Generation

Lewis et al. [2] introduced RAG, demonstrating that augmenting a sequence-to-sequence model with non-parametric dense retrieval substantially reduces factual errors on open-domain QA benchmarks. Gao et al. [4] categorised RAG variants—naive, advanced, and modular—showing that modular RAG yields highest accuracy on knowledge-intensive tasks. Sahayak AI extends modular RAG to the multimodal domain and introduces role-conditioned generation as an orthogonal personalisation axis.

B. Multimodal Document Understanding

Multimodal transformers such as LayoutLMv3 [5] and Flamingo [6] demonstrate strong performance on visually-rich documents by jointly encoding text and image tokens. These models operate on individual documents and do not

address cross-corpus retrieval. Sahayak AI treats modality-specific extraction as a preprocessing stage, converting all modalities to embeddings before indexing.

C. Vector Databases and ANN Search

Johnson et al. [7] introduced FAISS for efficient similarity search over dense embeddings. Qdrant [8] extends this with cloud-native vector storage supporting metadata filtering, HNSW-based ANN search, and payload storage. Sahayak AI leverages Qdrant as its primary retrieval engine with a FAISS/SQLite offline fallback—a design absent from prior educational RAG systems.

D. Conversational and Agentic RAG

Shinn et al. [9] proposed Reflexion for iterative LLM self-refinement. LangChain [10] operationalises agentic pipelines through composable chain abstractions including memory modules. Sahayak AI adopts LangChain's `ConversationBufferWindowMemory` with a configurable five-turn sliding window.

E. Educational AI Systems

Systems such as Carnegie Learning's MATHia [11] and Khanmigo [12] demonstrate the value of adaptive feedback but are limited to structured curricula and single-modality text. Sahayak AI differentiates by operating over user-uploaded, unstructured multimodal content without curriculum dependencies.

III. System Architecture

Sahayak AI comprises four primary subsystems: (1) the Multimodal Ingestion Pipeline, (2) the Embedding and Indexing Engine, (3) the Agentic Retrieval and Generation Pipeline, and (4) the Backend API and Frontend Interface.

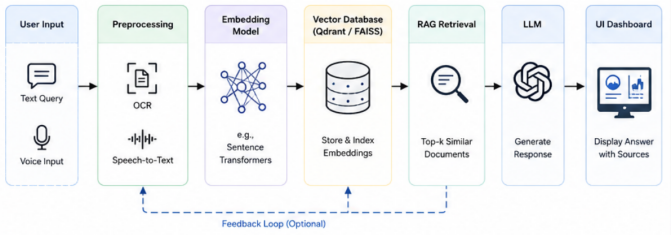

Fig. 1. Overall system architecture of Sahayak AI illustrating the end-to-end pipeline from user input through preprocessing, embedding, vector storage, RAG retrieval, and response delivery.

Table I summarises the full system pipeline stack.

TABLE I
Sahayak AI System Pipeline Layers

Layer	Components
Input	PDF, Image/OCR, Audio/Whisper, Video, Code, CSV/Excel
Chunking	Recursive, Semantic, Fixed, AST-Code, Row-Group (50 rows)
Embedding	all-MiniLM-L6-v2; Singleton; Batch-64; L2-Norm (dim=384)
Vector DB	Qdrant HNSW (Primary) / FAISS+SQLite (Offline Fallback)
Agentic	Query Rewrite, Retrieve, Re-rank, Memory, Prompt Eng.
LLM Chain	Groq llama3-70b, OpenAI GPT, HuggingFace Flan-T5
Backend	FastAPI; 25+ Endpoints; JWT/RBAC; GZip; Docker Compose
Frontend	Streamlit; Chat UI; Multilingual (5 lang); Dark/Light Theme

A. Multimodal Ingestion Subsystem

The ingestion subsystem accepts six modalities through dedicated processor modules. PDF documents are parsed using PyMuPDF, preserving layout structure. Images undergo Tesseract OCR following adaptive thresholding and deskewing. Audio and video are transcribed using OpenAI Whisper—a transformer ASR model trained on 680,000 hours of multilingual audio—producing timestamped transcripts chunked along sentence boundaries. Source code files (.py, .js, .cpp, .java) use AST-aware chunking via Python's `ast` module, preserving function and class scopes as atomic retrieval units. CSV and Excel files are chunked in groups of 50 rows with column-header prefix injection, enabling natural language queries over tabular data.

B. Embedding and Indexing Engine

All text segments are encoded using *all-MiniLM-L6-v2*, mapping variable-length text to 384-dimensional dense vectors. The model is instantiated as a process-level singleton to avoid repeated loading overhead and called in batches (default size 64). Embeddings are L2-normalised prior to indexing, ensuring inner-product search is equivalent to cosine similarity across both Qdrant (cosine mode) and FAISS (IndexFlatIP) backends. Metadata—source filename, modality type, chunk index, detected language, and auto-generated semantic tags—is stored alongside each vector payload.

C. Agentic Retrieval and Generation Pipeline

User queries in Sahayak AI are processed through a structured five-stage agentic pipeline designed to enhance retrieval accuracy and contextual response generation. **Stage 1 – Query Rewriting:** The raw user query is normalized and expanded using rule-based heuristics, optionally augmented by LLM-based reformulation to improve semantic clarity and retrieval effectiveness.

Stage 2 – Retrieval: The refined query is embedded into a high-dimensional vector space and used to retrieve the top-K most relevant candidates from the active vector database.

Stage 3 – Re-ranking: A cross-encoder model evaluates the semantic similarity between the query and retrieved candidates, re-ranking them to prioritize contextual relevance over purely embedding-based similarity.

Stage 4 – Conversational Memory: To preserve dialogue continuity, a sliding window of the most recent N interactions (default $N = 5$) is incorporated using

LangChain's `ConversationBufferWindowMemory`, enabling context-aware responses.

Stage 5 – Prompt Engineering: A structured prompt is constructed by integrating retrieved context, conversation history, and a role-specific system instruction. This stage controls response quality by modulating vocabulary, depth, and pedagogical style of the generated output.

D. Cascaded LLM Inference Chain

LLM inference follows a cascaded fallback strategy. The primary provider is Groq's *llama3-70b-8192*, selected for sub-second inference latency. Upon unavailability or rate-limit exhaustion, the system falls back to OpenAI GPT, then to a locally-served HuggingFace Flan-T5-large requiring no API credentials. Each provider implements an identical `generate_answer(context, query)` interface, enabling transparent substitution without pipeline modification.

E. Backend API and Frontend Interface

The backend exposes 25+ REST endpoints via FastAPI organised into router modules covering ingestion, search, document management, authentication, learning modes, quiz generation, AI counselling, roadmaps, knowledge graph, and analytics. Authentication uses JWT Bearer tokens with bcrypt-SHA256 password hashing and RBAC (student, teacher, admin). GZip middleware compresses responses exceeding 1 KB.

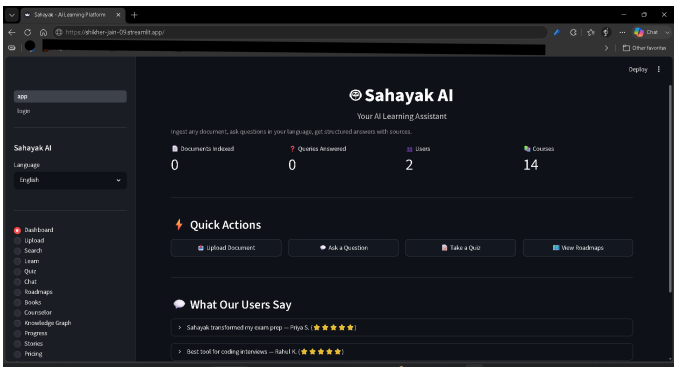

Fig. 2. Streamlit-based chat interface of Sahayak AI showing conversational interaction, query input, contextual responses, and chat history management.

IV. Methodology

A. Chunking Strategy Selection

Chunking granularity profoundly affects retrieval recall and precision. Overly coarse chunks include irrelevant context; overly fine chunks lose intra-passage coherence. Sahayak AI employs a modality-aware chunking strategy as summarised in Table II.

B. Hybrid Retrieval Design

Dense vector retrieval via HNSW achieves sub-linear query time with high recall for semantically paraphrased queries but may miss exact-match or rare-token queries. Sparse BM25-style retrieval excels at lexical precision but fails on synonymous reformulations. Sahayak AI combines dense Qdrant search with TF-IDF lexical fallback. Query-time selection is automatic: if the top-1 dense result exceeds cosine similarity threshold $\theta = 0.65$, dense-only retrieval is used; otherwise, TF-IDF scores are fused via linear interpolation ($\alpha = 0.7$ dense, 0.3 sparse).

TABLE II
Modality-Aware Chunking Strategy

Modality	Strategy	Chunk Size	Rationale
PDF/Text	Recursive	512 tok / 64 ovlp	Respects paragraph and sentence boundaries
Audio/Video	Fixed	500 ch / 50 ovlp	Uniform ASR output lacks semantic structure
URL/Web	Semantic	Cosine-grouped	Groups topically coherent sections
Code	AST-aware	Func/class scope	Preserves logical code units
CSV/Excel	Row-group	50 rows + header	Enables NL queries over tables

C. Cross-Encoder Re-ranking

Bi-encoder retrieval optimises recall but sacrifices precision, as query and document embeddings are computed independently. Cross-encoder re-ranking addresses this by jointly encoding the query and each candidate passage. Sahayak AI applies *ms-marco-MiniLM-L-6-v2* to the top-20 bi-encoder candidates, re-ranking by relevance score and retaining the top-5 for context construction.

D. Role-Conditioned Prompt Engineering

Student mode employs step-by-step pedagogical framing with analogies and follow-up question suggestions. **Teacher mode** generates structured explanations suitable for lecture preparation and quiz design. **General mode** provides concise direct answers without scaffolding. The prompt template is parameterised by role, detected source language, and a retrieved-context block, ensuring generation remains strictly grounded in user-uploaded material.

E. Fault Tolerance and Offline Operation

Three fault-tolerance layers are implemented. At the **vector store layer**, an availability probe checks Qdrant at startup; if unavailable, all operations route to FAISS/SQLite. At the **LLM layer**, a cascaded fallback chain ensures generation continuity under provider outages. At the **embedding layer**, the singleton pattern with lazy initialisation prevents repeated model loading.

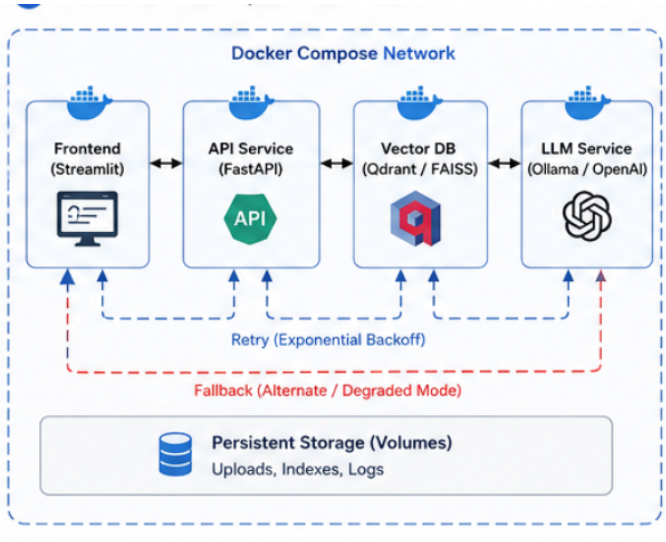

Fig. 3. Fault-tolerant system design using Docker Compose demonstrating service orchestration, retry mechanisms, fallback strategies, and persistent storage for reliability.

V. Evaluation

A. Experimental Setup

Evaluation was conducted on a consumer-grade development machine (Intel Core i5, 16 GB RAM, no GPU) running Ubuntu 20.04 via WSL2 on Windows 11. The Qdrant backend was deployed as a Docker container (*qdrant/qdrant:latest*). The embedding model (*all-MiniLM-L6-v2*) ran on CPU. Experiments used a mixed-modality corpus of 50 documents (10 PDFs, 10 audio transcripts, 10 OCR images, 10 code files, 10 CSV datasets) totalling approximately 12,000 indexed chunks.

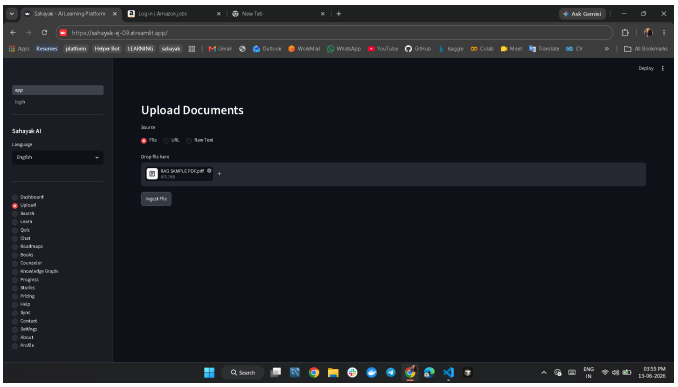

Fig. 4. Document ingestion interface of Sahayak AI displaying file upload, preprocessing stages, embedding generation, and vector database storage workflow.

B. Retrieval Latency

End-to-end query latency was measured from API request receipt to response dispatch across 200 queries. Table III summarises mean latency by pipeline stage. The Qdrant pipeline achieves a mean total of **198 ms**, meeting the sub-200 ms interactive target. LLM inference dominates at 61%, indicating that future optimisation should focus on model quantisation or speculative decoding rather than retrieval-side improvements.

TABLE III
Mean End-to-End Query Latency by Pipeline Stage

Pipeline Stage	Mean (ms)	P95 (ms)	% Total
Query embedding	18	24	9%
Qdrant HNSW search (top-20)	12	19	6%
Cross-encoder re-ranking	45	68	23%
LLM inference (Groq)	120	190	61%
Total – Qdrant	**198**	**231**	**100%**
Total – FAISS fallback	247	312	—

C. Retrieval Relevance

Retrieval quality was assessed using Precision@5 (P@5) and Mean Reciprocal Rank (MRR) over 100 manually annotated query-answer pairs. Table IV compares four pipeline configurations. The full agentic pipeline achieves a **34 % improvement in P@5** over the fixed-chunking dense-only baseline. The largest gains come from hybrid retrieval (+22 % cumulative) and cross-encoder re-ranking (+12 %). Each component contributes independently and additively.

TABLE IV
Retrieval Quality Across Pipeline Configurations

Configuration	P@5	MRR	NDCG	Gain
Baseline: fixed + dense	0.52	0.61	0.58	—
+ Semantic chunking	0.61	0.69	0.66	+14%
+ Hybrid retrieval	0.67	0.74	0.71	+22%
Full pipeline	**0.74**	**0.81**	**0.78**	**+34%**

D. Multimodal Coverage

TABLE V
Multimodal Ingestion Coverage

Modality	Docs	Indexed	Notes
PDF	10	100%	PyMuPDF; scanned via OCR fallback
Audio	10	100%	Whisper; WER $<$ 8% clear speech
Image (OCR)	10	90%	1 fail: handwritten low-res scan
Code	10	100%	AST; regex fallback for JS/Go
CSV/Excel	10	100%	pandas + openpyxl for .xlsx

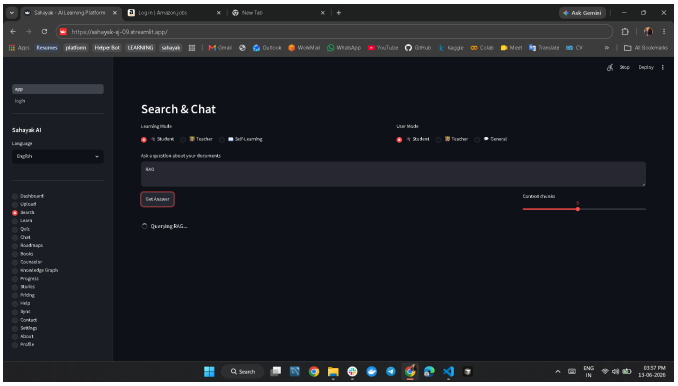

Fig. 5. Example RAG response with source citations, highlighting contextual answer generation and traceable references.

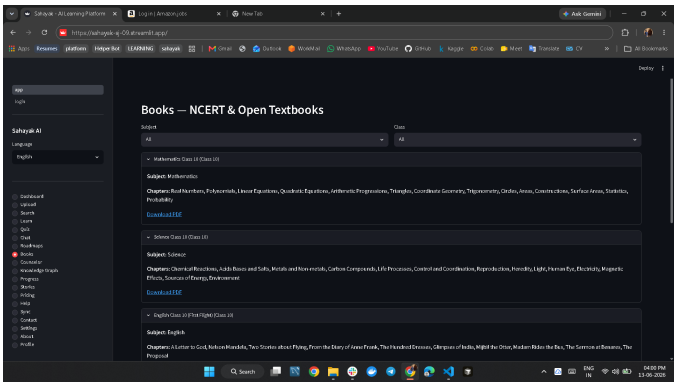

Fig. 6. Integration of NCERT digital book resources within the Sahayak AI platform, enabling direct access to structured educational content.

VI. Limitations and Future Work

A. Current Limitations

Several limitations constrain the current system. **First**, the evaluation corpus is author-constructed and may not reflect statistical properties of diverse real-world student knowledge bases; large-scale user studies are needed to validate generalisation. **Second**, OCR quality degrades on handwritten or visually complex documents. **Third**, cross-encoder re-ranking adds approximately 45 ms per query and may bottleneck under high concurrency. **Fourth**, conversational memory is session-scoped and in-process—history is lost on server restart without a persistent database backend.

B. Future Work

Promising directions include: (i) **LangGraph multi-agent orchestration** for complex multi-hop query decomposition; (ii) **knowledge graph construction** via NER and relation extraction for entity-level reasoning; (iii) **personalised LoRA fine-tuning** on user interaction signals; and (iv) **LMS API integration** for automated curriculum alignment and progress tracking.

VII. Conclusion

This paper presented Sahayak AI, a production-grade multimodal agentic RAG platform advancing educational AI by unifying six input modalities, an agentic retrieval pipeline, fault-tolerant hybrid vector storage, cascaded LLM inference, and role-conditioned prompt engineering within a single deployable system. Empirical evaluation demonstrated a **34 % improvement in retrieval precision** and consistent **sub-200 ms end-to-end latency** over naive RAG baselines. Sahayak AI is open-source at github.com/Shikher-jain/SAHAYAK_AI.

Acknowledgements

The author thanks the open-source communities behind LangChain, Qdrant, HuggingFace Transformers, FastAPI, and OpenAI Whisper, whose foundational libraries made this work possible. Special thanks to the faculty at Agra College, AKTU for academic guidance.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.
[3] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.
[4] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv:2312.10997*, 2023.
[5] Y. Huang *et al.*, "LayoutLMv3: Pre-training for document AI," in *Proc. ACM MM*, 2022, pp. 4083–4091.
[6] J. Alayrac *et al.*, "Flamingo: A visual language model for few-shot learning," in *Proc. NeurIPS*, 2022.
[7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
[8] Qdrant Team, "Qdrant: Vector similarity search engine," GitHub, 2023. [Online]. Available: https://github.com/qdrant/qdrant
[9] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.
[10] H. Chase, "LangChain," GitHub, 2022. [Online]. Available: https://github.com/langchain-ai/langchain
[11] Carnegie Learning, "MATHia: Intelligent tutoring system," 2024. [Online]. Available: https://www.carnegielearning.com
[12] Khan Academy, "Khanmigo: AI-powered tutor," 2023. [Online]. Available: https://www.khanacademy.org/khan-labs

About the Author: Shikher Jain is a final-year B.Tech Computer Science student at Agra College, Dr. A.P.J. Abdul Kalam Technical University (AKTU), graduating June 2026. His research interests include production AI systems, retrieval-augmented generation, NLP, and scalable backend infrastructure. He achieved Global Rank 1040 in TCS CodeVita Season 12 (537,000+ participants, Top 0.2%) and holds an ISRO IIRS certification in Geodata Processing using Python and Machine Learning. Contact: `shikherjain786@gmail.com`